

6-3 图的遍历

1.广度优先搜索BFS

代码表示

```
bool visited[MVNum]; //辅助数组
void BFS_AM(AMGraph G, int v){ // 使用邻接矩阵表示的图的广度优先搜索算法
    InitQueue(Q); // 初始化队列
    cout << v;
    visited[v] = true; // 标记顶点v为已访问
    EnQueue(Q, v); // 顶点v入队
    while(!QueueEmpty(Q)){ // 队列非空
        DeQueue(Q, u); // 出队
        for(int w = 0; w < G.vexnum; w++){
            if((G.arcs[u][w] != 0) && (!visited[w])){
                cout << w; // 输出当前访问的顶点编号w
                visited[w] = true; // 标记顶点w为已访问
                EnQueue(Q, w); // 顶点w入队
            }
        }
    }
}
```

时间复杂度

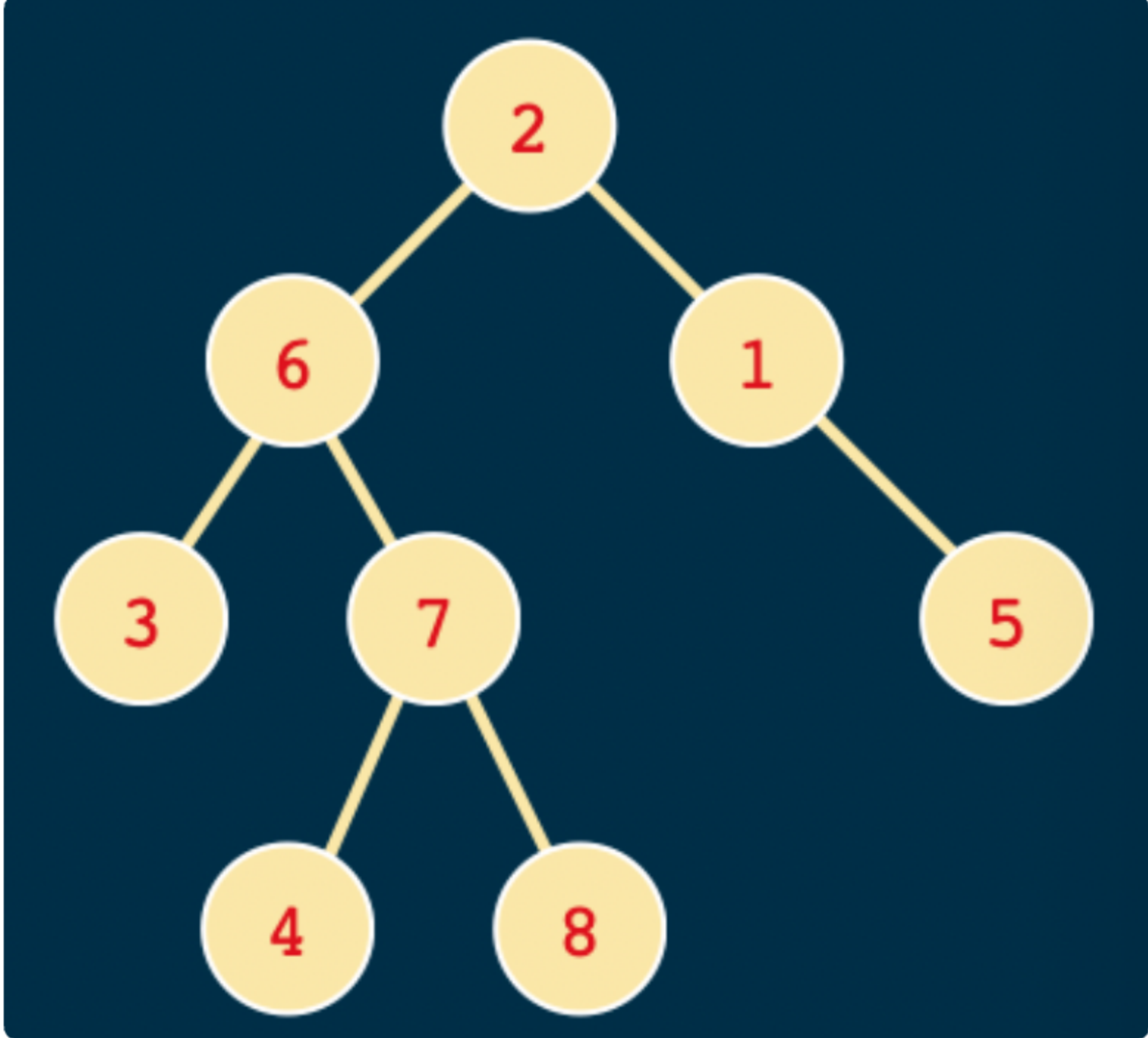
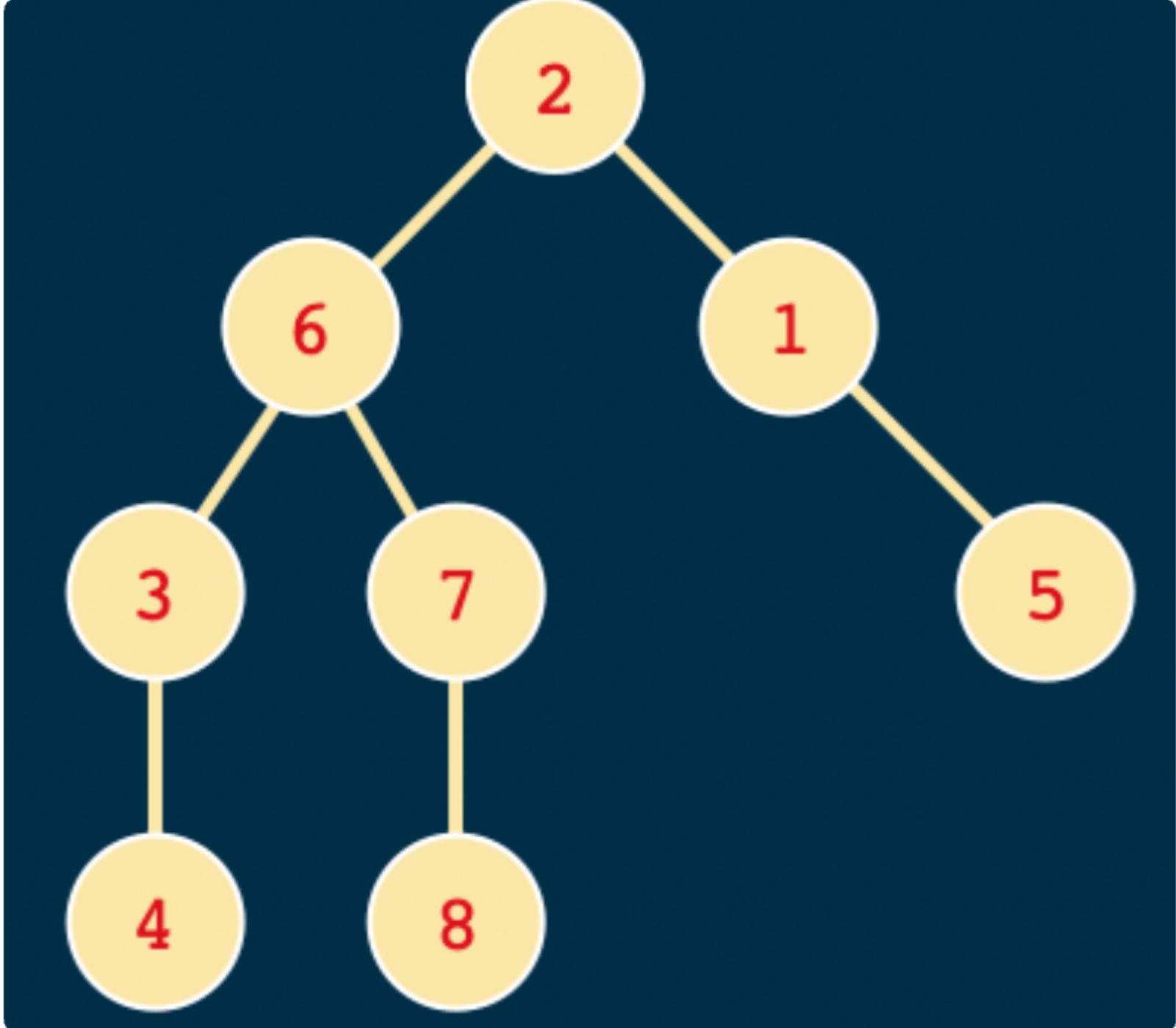
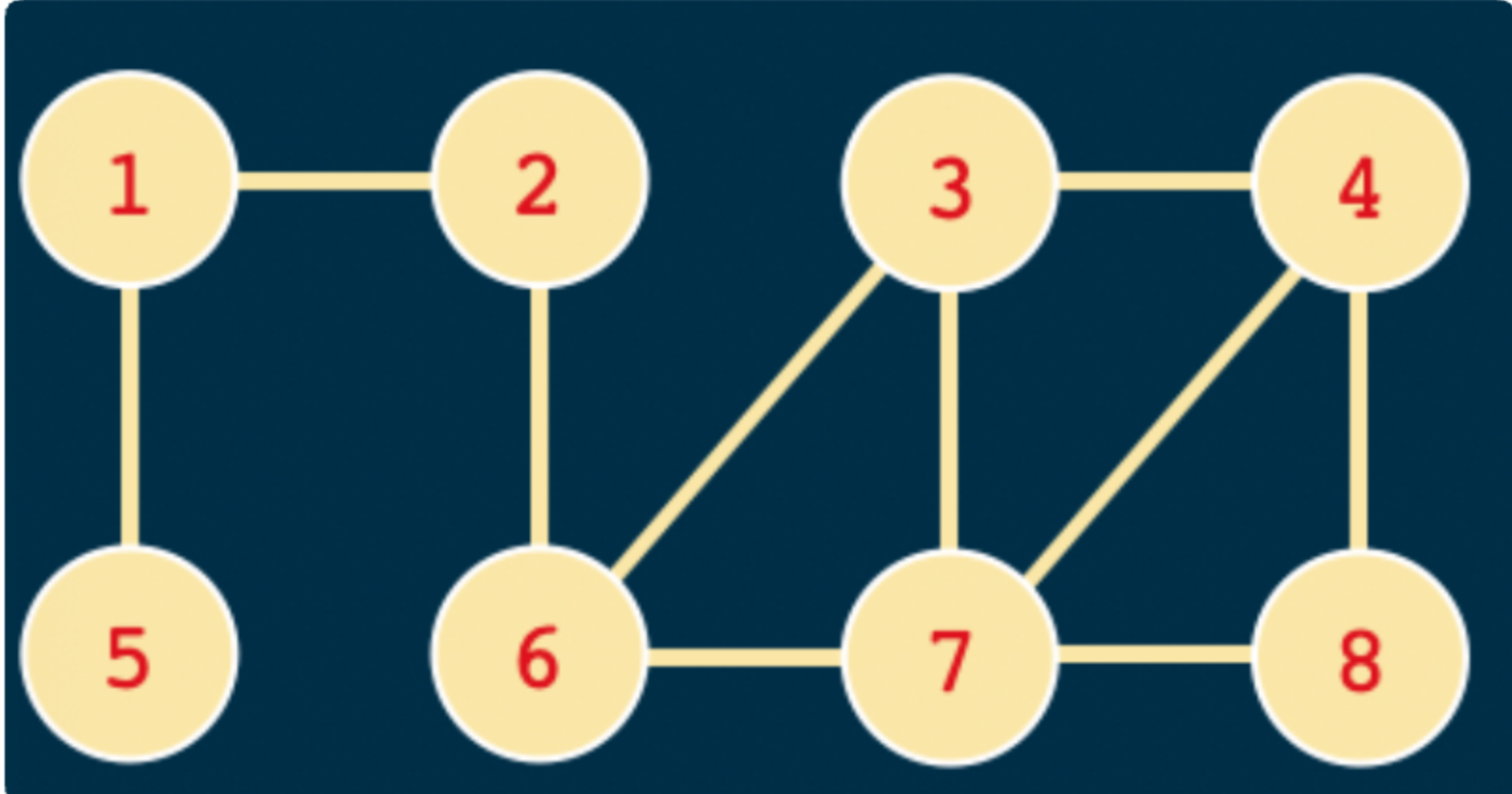
- 邻接矩阵 $O(n^2)$
- 邻接表 $O(n+e)$

最坏情况空间复杂度

$O(n)$ (队列)

无向连通图只需要一次BFS_AM就可以全图遍历，非连通图需要靠循环n次，对未访问节点BFS， $n=(\text{连通分量数})$

图例



邻接矩阵下，同一源点出发得到的广度优先生成树唯一，邻接表下，因为邻接表表达不唯一，广度优先生成树也不唯一

2.深度优先搜索DFS

代码表示

```
bool visited[MVNum]; // 全局访问标记数组
void DFS_AL(ALGraph G, int v) { // 使用邻接表表示的图的深度优先搜索算法
    cout << v; // 输出当前访问的顶点编号v
    visited[v] = true; // 标记顶点v为已访问
    p = G.vertices[v].firstarc; // 获取顶点v的邻接表的头指针
    while (p != NULL) { // 循环遍历顶点v的所有邻接点
        w = p->adjvex; // 获取当前邻接顶点的编号
        if (!visited[w]) // 移动到顶点v的下一个邻接点
            DFS_AL(G, w);
        p = p->nextarc;
    }
}
```

时间复杂度

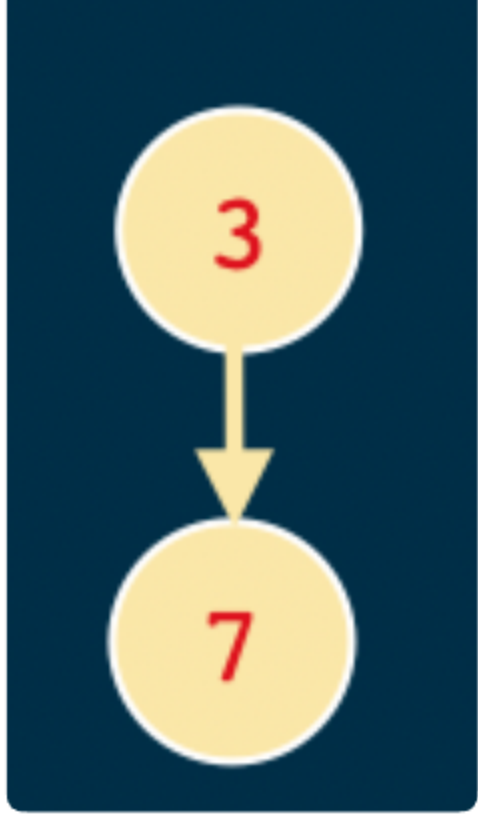
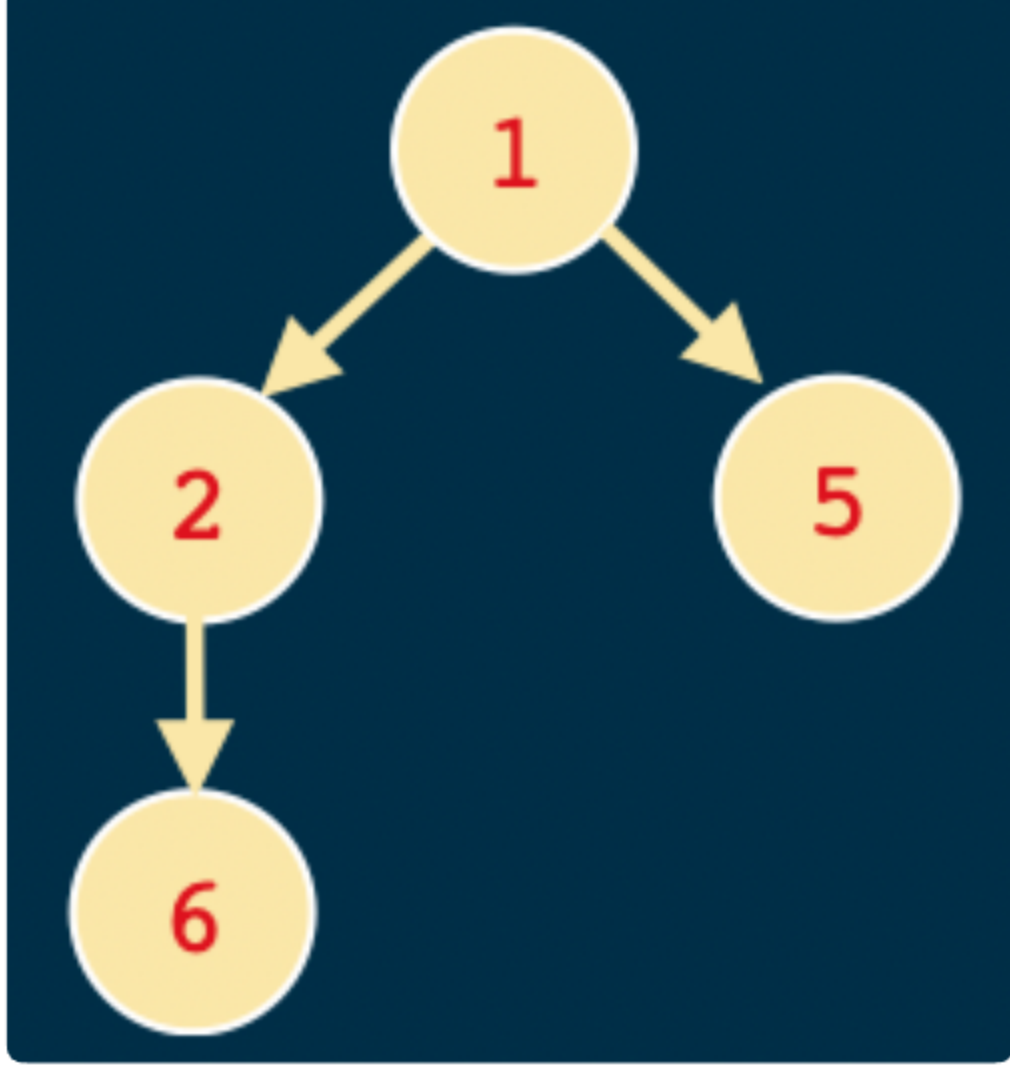
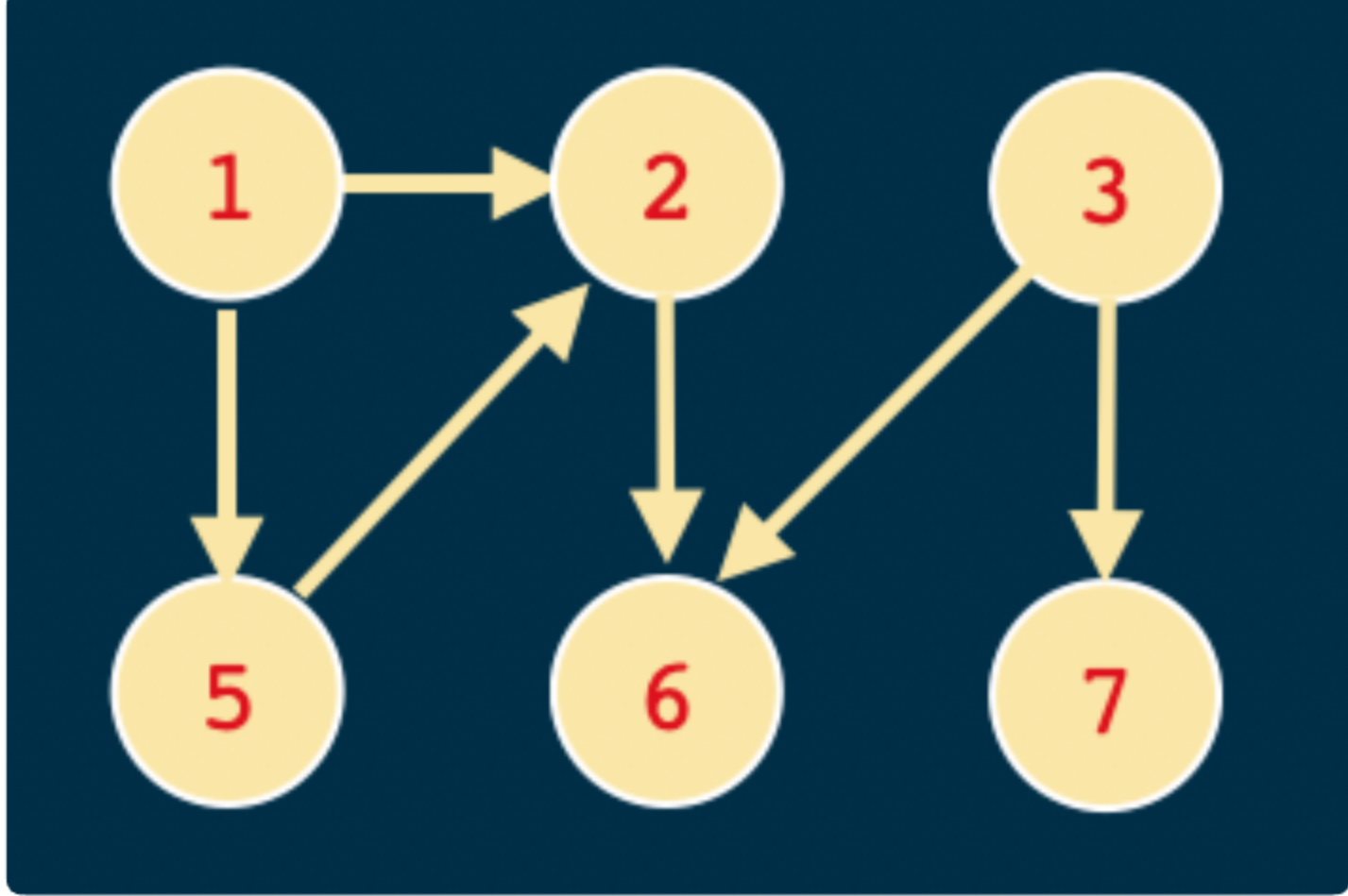
- 邻接矩阵 $O(n^2)$
- 邻接表 $O(n+e)$

最坏情况空间复杂度

$O(n)$ (递归栈深度)

无向连通图只需要一次DFS_AL就可以全图遍历，非连通图需要靠循环n次，对未访问节点DFS， $n=(\text{连通分量数})$

图例



和BFS相同，邻接矩阵下，同一源点出发得到的深度优先生成树唯一，邻接表下，因为邻接表表达不唯一，深度优先生成树也不唯一